Tarefa 15: Sobreposição de Computação e Comunicação

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

O objetivo desta tarefa é medir quanto se ganha ao sobrepor comunicação e computação em um programa MPI. O problema escolhido é a difusão de calor em uma barra 1D dobrada em anel (contorno periódico), resolvida por diferenças finitas explícitas. A barra é dividida entre os processos, e a cada passo de tempo vizinhos precisam trocar suas células de borda.

Foram implementadas três versões da troca de bordas:

- **V1:** `MPI_Send` / `MPI_Recv` (bloqueante);
- **V2:** `MPI_Isend` / `MPI_Irecv` e `MPI_Wait`;
- **V3:** igual à V2, mas calculando os pontos internos enquanto a comunicação acontece, com `MPI_Test`.

Ferramentas utilizadas: MPI e o supercomputador NPAD.

2 Metodologia

Decomposição e células fantasmas. Cada processo guarda um trecho contíguo da barra, com n células reais mais duas células fantasmas (*ghost cells*): $u[0]$ recebe a última célula real do vizinho da esquerda e $u[n+1]$ recebe a primeira célula real do vizinho da direita. Como a barra é um anel, o processo 0 é vizinho do último processo. Duas tags distinguem o sentido das mensagens (direita e esquerda), o que também funciona com apenas 2 processos.

As três versões.

- **V1:** a troca é feita em duas fases em cadeia (primeiro para a direita, depois para a esquerda). O processo 0 envia primeiro para quebrar o ciclo e evitar *deadlock*; os demais recebem primeiro e depois enviam.
- **V2:** os quatro `Isend`/`Irecv` são postados de uma vez, o programa espera todos com `MPI_Wait` e só então calcula todas as células.
- **V3:** após postar as operações, calcula os pontos internos (que não dependem do halo) em blocos de 100 células, chamando `MPI_Test` entre os blocos. Depois espera o fim da comunicação e calcula as duas células de borda.

Experimentos. Os testes rodaram no NPAD com 2 nós, com 2, 4 e 8 processos distribuídos entre eles (1, 2 e 4 processos por nó), e com $N = 1000$, 5000 e 10000 pontos. O tempo de cada versão foi medido com `MPI_Wtime` e corresponde ao maior tempo entre os processos, contando só cálculo e comunicação.

3 Resultados

A Figura 1 mostra a evolução da temperatura obtida pelas três versões. Os resultados são visualmente idênticos, o que indica que a troca de células fantasmas está correta em todas elas: o calor se espalha a partir do centro e a barra tende ao equilíbrio.

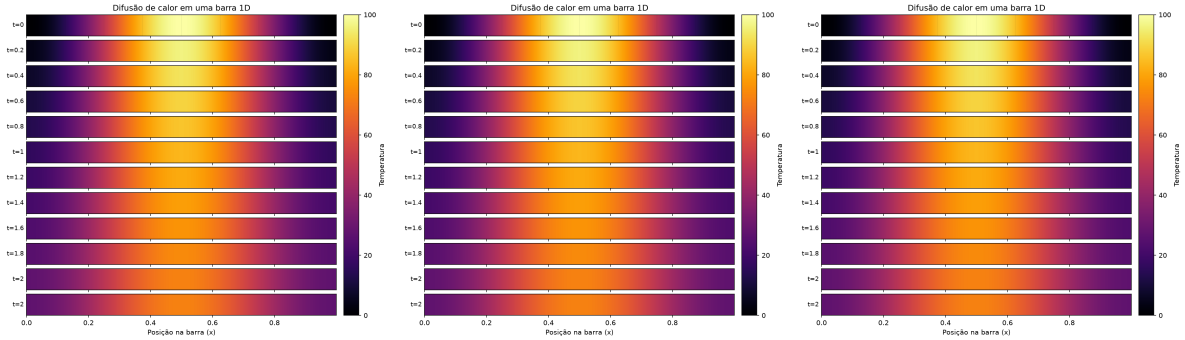


Figura 1: Evolução da temperatura na barra: V1, V2 e V3 (da esquerda para a direita).

A Tabela 1 reúne os tempos medidos e o ganho de V2 e V3 em relação à V1.

N	Proc.	V1 (s)	V2 (s)	V3 (s)	V1/V2	V1/V3
1000	2	0,205	0,143	0,136	1,43	1,50
1000	4	0,319	0,127	0,103	2,51	3,10
1000	8	0,626	0,145	0,118	4,31	5,29
5000	2	11,71	10,13	12,52	1,16	0,94
5000	4	10,91	6,26	7,24	1,74	1,51
5000	8	18,13	5,90	6,41	3,07	2,83
10000	2	80,42	73,95	96,59	1,09	0,83
10000	4	60,50	40,93	51,07	1,48	1,18
10000	8	88,25	39,47	44,66	2,24	1,98

Tabela 1: Tempos de execução e ganho sobre a V1 (valores acima de 1 indicam que a versão é mais rápida que a V1).

4 Análise

Impacto das versões no tempo. A escolha da versão muda bastante o tempo de execução. Para $N = 1000$ e 8 processos, a V1 leva 0,63 s contra 0,15 s da V2 e 0,12 s da V3. Com 8 processos, V2 e V3 foram de 2 a 5 vezes mais rápidas que a V1 em todos os tamanhos testados.

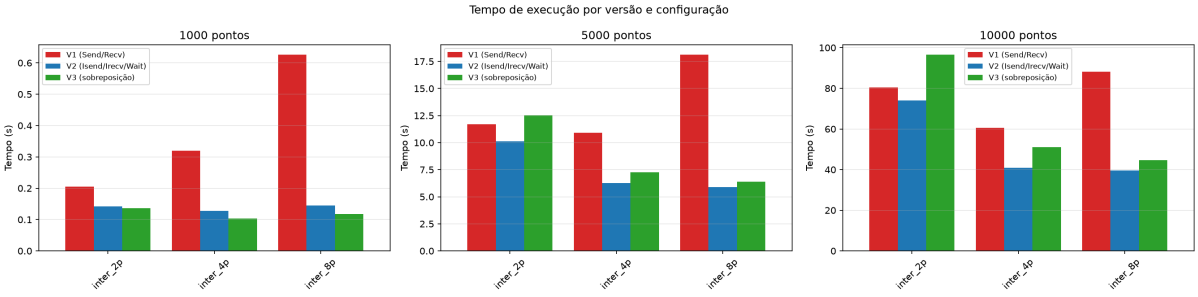


Figura 2: Tempo de execução de cada versão por configuração.

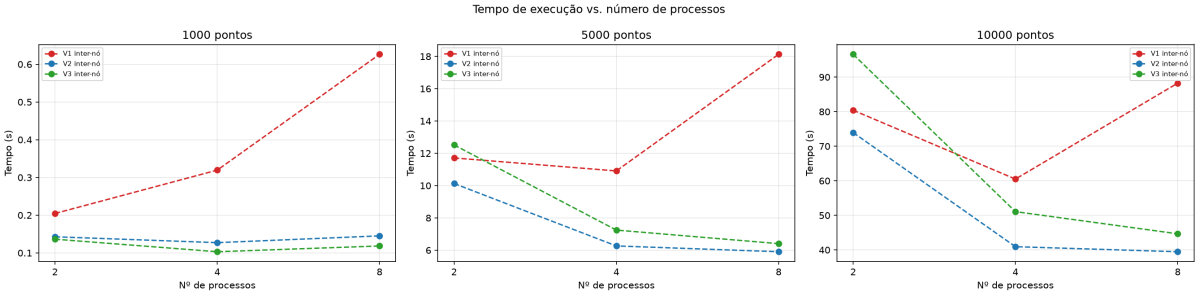


Figura 3: Tempo de execução em função do número de processos.

A V1 piora com mais processos. Mesmo com N fixo, o tempo da V1 tende a crescer quando se aumentam os processos (Figura 3). Com $N = 1000$, ele sobe de 0,20 s para 0,63 s ao passar de 2 para 8 processos, embora cada processo tenha menos células para calcular. A causa é o bloqueio: na V1 cada processo só envia depois de receber do anterior, então a troca de bordas vira uma cadeia e o tempo de comunicação cresce com o número de processos, enquanto os demais ficam esperando. Nos tamanhos maiores o efeito é mascarado pelo cálculo, mas continua visível (por exemplo, $N = 10000$ passa de 60,5 s com 4 processos para 88,3 s com 8).

Impacto da V2 e da V3. Na V2 as quatro operações são postadas ao mesmo tempo, então as trocas de todos os pares acontecem em paralelo e o custo de comunicação deixa de crescer com o número de processos. Com $N = 1000$ o tempo fica praticamente constante (0,13 a 0,15 s), e com N maior ele cai conforme se adicionam processos. Já a V3 teve comportamento misto em relação à V2, discutido a seguir.

Ganho com a sobreposição. Para $N = 1000$, a sobreposição compensou: a V3 foi cerca de 19% mais rápida que a V2 com 4 e 8 processos (e 4% com 2), e é onde a comunicação pesa mais no tempo total. Para $N = 5000$ e $N = 10000$, porém, a V3 ficou mais lenta que a V2 (até 31% mais lenta, em $N = 10000$ com 2 processos).

Uma explicação provável é o custo das chamadas de `MPI_Test`: com blocos de 100 células, são 4 chamadas por bloco a cada passo, e esse número cresce com o tamanho do trecho de cada processo. Ou seja, a sobreposição só vale a pena quando o que ela esconde é maior que o custo de testar a comunicação, e o tamanho do bloco é o parâmetro que controla esse equilíbrio.

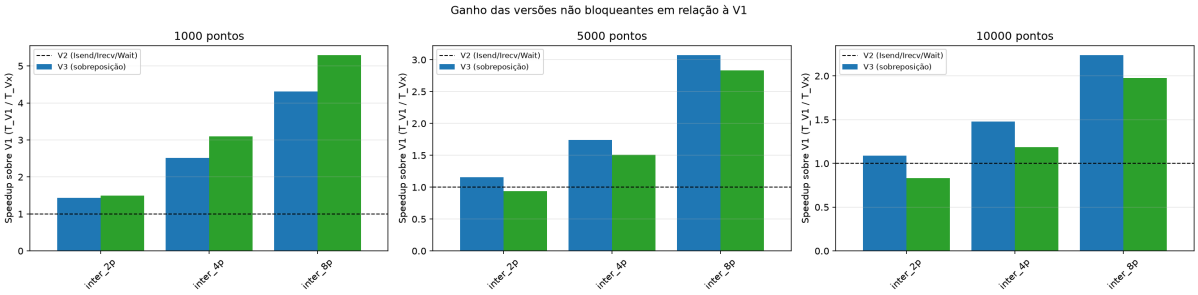


Figura 4: Ganho de V2 (azul) e V3 (verde) em relação à V1.

Limitações. Cada configuração foi medida uma só vez, então diferenças pequenas (como V3 contra V1 em $N = 5000$ com 2 processos) podem ser ruído. Os testes registraram avisos do Open MPI sobre a placa InfiniBand, e a comunicação entre nós pode ter usado um caminho mais lento que o ideal, o que também afeta os tempos absolutos.

5 Conclusão

- A troca bloqueante da V1 não escala: seu tempo cresce com o número de processos mesmo para N fixo, por causa da espera em cadeia.
- Comunicação não bloqueante (V2) resolve esse problema e foi, no geral, a versão mais regular, com ganhos de até 4,3 vezes sobre a V1.
- A sobreposição com `MPI_Test` (V3) trouxe o melhor tempo nos casos pequenos ($N = 1000$), mas perdeu para a V2 nos maiores, mostrando que o ganho depende de ajustar a frequência dos testes.
- Como continuação, vale testar blocos maiores (ou menos chamadas de `MPI_Test`) na V3 e repetir as medições para obter médias.

A Código-Fonte

```c
/*
 * Simulação de difusão de calor em uma barra 1D DOBRADA EM ANEL
 * Método: Diferenças Finitas Explícito (FTCS - Forward Time, Centered Space)
 * Condição de contorno: PERIÓDICA (o último ponto évizinho do primeiro)
 *
 * Equação resolvida: du/dt = alpha * d^2u/dx^2
 *
 * Condição inicial: perfil em cosseno, mais quente no centro da barra
 * e mais frio nas extremidades e como as extremidades se encontram
 * (formando o anel), a temperatura já "fecha" nesse ponto.
 *
 * Compilar: gcc barra_1d.c -o barra_1d -lm
 * Executar: ./barra_1d
 *
 * Saída: arquivo "temperaturas.csv" com a temperatura ao longo do anel
 *        em diferentes instantes de tempo (uma linha por instante salvo).
 */


#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>
#include <sys/stat.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

/* Parâmetros de simulação */
#define N_PONTOS 1000        /* Número de pontos na barra */
#define COMPRIMENTO 1.0      /* Comprimento da barra (m) */
#define ALPHA 0.01           /* Difusividade térmica (m^2/s) */
#define T_TOTAL 2.0          /* Tempo total de simulação (s) */
#define TEMP_BASE 0.0        /* temperatura nas bordas/base (ºC) */
#define TEMP_CENTRO 100.0    /* temperatura no centro da barra (ºC) */
#define N_SALVAS 10          /* quantos instantes de tempo salvar no CSV */

#define TAG_DIREITA 0
#define TAG_ESQUERDA 1
#define BLOCO 100            /* celulas calculadas entre chamdas de MPI_Teste (V3) */

typedef struct {
    int rank, size;
    int esquerda, direita;   /* vizinhos no anel */
    int n_total;             /* pontos na barra inteira */
    int n;                   /* pontos no processo atual */
    int offset;              /* indice global da 1 celula real */
    double dx, dt, r;
    int n_passos, intervalo_salvar;
    int *counts, *displs;    /* usados no Gatherv (so rank 0) */
    double *global;          /* buffer da barra inteira (so rank 0) */
} Sim;

/**
 * FUNCOES AUXILIARES
```

```
 */

static void atualiza(const double *u, double *un, int ini, int fim, double r)
{
    for (int i = ini; i <= fim; i++)
        un[i] = u[i] + r * (u[i + 1] - 2.0 * u[i] + u[i - 1]);
}

static void condicao_inicial(const Sim *s, double *u)
{
    for (int i = 1; i < s->n; i++) {
        double x = (s->offset + i - 1) * s->dx;
        double fase = 2.0 * M_PI * (x - COMPRIMENTO / 2.0) / COMPRIMENTO;
        u[i] = TEMP_BASE + (TEMP_CENTRO - TEMP_BASE) * (cos(fase) + 1.0) / 2.0;
    }
}

static void salvar_linha(const Sim *s, FILE *f, double t, const double *u)
{
    MPI_Gatherv(&u[1], s->n, MPI_DOUBLE, s->global, s->counts, s->displs,
                MPI_DOUBLE, 0, MPI_COMM_WORLD);
    if (s->rank == 0) {
        fprintf(f, "%.6f", t);
        for (int i = 0; i < s->n_total; i++) fprintf(f, ",%.6f", s->global[i]);
        fprintf(f, "\n");
    }
}

static FILE *abrir_csv(const Sim *s, const char *nome)
{
    if (s->rank != 0) return NULL;
    FILE *f = fopen(nome, "w");

    if (!f) {
        fprintf(stderr, "Erro ao criar %s\n", nome);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    fprintf(f, "tempo");
    for (int i = 0; i < s->n_total; i++) fprintf(f, ",%.6f", i * s->dx);
    fprintf(f, "\n");

    return f;
}

typedef void (*PassoFn)(const Sim *, double *, double *);

static double executar(const Sim *s, const char *arquivo, PassoFn passo_fn)
{
    double *u = calloc(s->n + 2, sizeof(double));
    double *un = calloc(s->n + 2, sizeof(double));

    condicao_inicial(s, u);
    FILE *f = abrir_csv(s, arquivo);
    salvar_linha(s, f, 0.0, u);

    MPI_Barrier(MPI_COMM_WORLD);
    double t_acum = 0.0;

```

```
115     for (int passo = 1; passo <= s->n_passos; passo++) {
116         double t0 = MPI_Wtime();
117         passo_fn(s, u, un);
118         double *tmp = u; u = un; un = tmp;
119         t_acum += MPI_Wtime() - t0;
120 
121         if (passo % s->intervalo_salvar == 0 || passo == s->n_passos)
122             salvar_linha(s, f, passo * s->dt, u);
123     }
124 
125     if (f) fclose(f);
126     free(u);
127     free(un);
128 
129     double t_max;
130     MPI_Reduce(&t_acum, &t_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
131 
132     return t_max;
133 }
134 
135 /**
136  * V1: SEND/RECV BLOQUEANTES
137  */
138 static void passo_v1(const Sim *s, double *u, double *un)
139 {
140     int n = s->n;
141     const int ultimo = s->size - 1;
142 
143     /* Fase 1: dados indo para direita (u[n] -> ghost u[0])*/
144     if (s->rank == 0) {
145         MPI_Send(&u[n], 1, MPI_DOUBLE, s->direita, TAG_DIREITA, MPI_COMM_WORLD);
146         MPI_Recv(&u[0], 1, MPI_DOUBLE, s->esquerda, TAG_DIREITA, MPI_COMM_WORLD,
147                  MPI_STATUS_IGNORE);
148     } else {
149         MPI_Recv(&u[0], 1, MPI_DOUBLE, s->esquerda, TAG_DIREITA, MPI_COMM_WORLD,
150                  MPI_STATUS_IGNORE);
151         MPI_Send(&u[n], 1, MPI_DOUBLE, s->direita, TAG_DIREITA, MPI_COMM_WORLD);
152     }
153 
154     /* Fase 2: dados indo para esquerda (u[1] -> ghost u[n+1])*/
155     if (s->rank == ultimo) {
156         MPI_Send(&u[1], 1, MPI_DOUBLE, s->esquerda, TAG_ESQUERDA, MPI_COMM_WORLD);
157         MPI_Recv(&u[n + 1], 1, MPI_DOUBLE, s->direita, TAG_ESQUERDA,
158                  MPI_COMM_WORLD, MPI_STATUS_IGNORE);
159     } else {
160         MPI_Recv(&u[n + 1], 1, MPI_DOUBLE, s->direita, TAG_ESQUERDA,
161                  MPI_COMM_WORLD, MPI_STATUS_IGNORE);
162         MPI_Send(&u[1], 1, MPI_DOUBLE, s->esquerda, TAG_ESQUERDA, MPI_COMM_WORLD);
163     }
164 
165     atualiza(u, un, 1, n, s->r);
166 }
167 
168 /**
169  * V2: COMUNICACAO NAO BLOQUEANTE, ESPERA ANTES DE CALCULAR
170  */
171 static void passo_v2(const Sim *s, double *u, double *un) {
172     int n = s->n;
```

```
173     MPI_Request req[4];
174
175     MPI_Irecv(&u[0], 1, MPI_DOUBLE, s->esquerda, TAG_DIREITA, MPI_COMM_WORLD, &req[0]);
176     MPI_Irecv(&u[n + 1], 1, MPI_DOUBLE, s->direita, TAG_ESQUERDA, MPI_COMM_WORLD, &req
            [1]);
177     MPI_Isend(&u[n], 1, MPI_DOUBLE, s->direita, TAG_DIREITA, MPI_COMM_WORLD, &req[2]);
178     MPI_Isend(&u[1], 1, MPI_DOUBLE, s->esquerda, TAG_ESQUERDA, MPI_COMM_WORLD, &req[3])
            ;
179
180     for (int k = 0; k < 4; k++) MPI_Wait(&req[k], MPI_STATUS_IGNORE);
181
182     atualiza(u, un, 1, n, s->r);
183 }
184
185 /**
186  * V3: SOBREPOSICAO
187  */
188 static void passo_v3(const Sim *s, double *u, double *un) {
189     int n = s->n;
190     MPI_Request req[4];
191     int flag;
192
193     MPI_Irecv(&u[0], 1, MPI_DOUBLE, s->esquerda, TAG_DIREITA, MPI_COMM_WORLD, &req[0]);
194     MPI_Irecv(&u[n + 1], 1, MPI_DOUBLE, s->direita, TAG_ESQUERDA, MPI_COMM_WORLD, &req
            [1]);
195     MPI_Isend(&u[n], 1, MPI_DOUBLE, s->direita, TAG_DIREITA, MPI_COMM_WORLD, &req[2]);
196     MPI_Isend(&u[1], 1, MPI_DOUBLE, s->esquerda, TAG_ESQUERDA, MPI_COMM_WORLD, &req[3])
            ;
197
198     /* Pontos internos, em blocos, testando a comunicação entre eles */
199     for (int ini = 2; ini <= n - 1; ini += BLOCO) {
200         int fim = ini + BLOCO - 1;
201         if (fim > n - 1) fim = n - 1;
202         atualiza(u, un, ini, fim, s->r);
203         for (int k = 0; k < 4; k++) MPI_Test(&req[k], &flag, MPI_STATUS_IGNORE);
204     }
205
206     /* Garante o término da comunicação antes de usar o halo */
207     for (int k = 0; k < 4; k++) MPI_Wait(&req[k], MPI_STATUS_IGNORE);
208
209     /* Pontos de borda (dependem das células fantasmas) */
210     atualiza(u, un, 1, 1, s->r);
211     atualiza(u, un, n, n, s->r);
212 }
213
214 /**
215  * FUNCOES PARA CADA VERSAO
216  */
217
218 double versao1(const Sim *s) { return executar(s, "dados/temperaturas_v1.csv",
        passo_v1); }
219 double versao2(const Sim *s) { return executar(s, "dados/temperaturas_v2.csv",
        passo_v2); }
220 double versao3(const Sim *s) { return executar(s, "dados/temperaturas_v3.csv",
        passo_v3); }
221
222 /**
223  * MAIN
```

```c
 */
int main(int argc, char **argv) {
    MPI_Init(&argc, &argv);

    Sim s;
    MPI_Comm_rank(MPI_COMM_WORLD, &s.rank);
    MPI_Comm_size(MPI_COMM_WORLD, &s.size);

    s.n_total = (argc > 1) ? atoi(argv[1]) : N_PONTOS_PADRAO;

    if (s.size < 2 || s.n_total < 2 * s.size) {
        if (s.rank == 0)
            fprintf(stderr, "Use pelo menos 2 processos e N_PONTOS >= 2*processos.\n");
        MPI_Finalize();
        return 1;
    }

    /* Vizinhos no anel */
    s.esquerda = (s.rank - 1 + s.size) % s.size;
    s.direita = (s.rank + 1) % s.size;

    /* Divisão da barra (o resto vai para os primeiros ranks) */
    int base = s.n_total / s.size, resto = s.n_total % s.size;
    s.n = base + (s.rank < resto ? 1 : 0);
    s.offset = s.rank * base + (s.rank < resto ? s.rank : resto);

    /* Parâmetros numéricos */
    s.dx = COMPRIMENTO / s.n_total;
    double dt_max = 0.5 * s.dx * s.dx / ALPHA;
    s.dt = 0.9 * dt_max;
    s.n_passos = (int)ceil(T_TOTAL / s.dt);
    s.dt = T_TOTAL / s.n_passos;
    s.r = ALPHA * s.dt / (s.dx * s.dx);
    s.intervalo_salvar = s.n_passos / N_SALVAS;
    if (s.intervalo_salvar < 1) s.intervalo_salvar = 1;

    /* Buffers do Gatherv (só no rank 0) */
    s.counts = s.displs = NULL;
    s.global = NULL;
    if (s.rank == 0) {
        mkdir("dados", 0777);
        s.counts = malloc(s.size * sizeof(int));
        s.displs = malloc(s.size * sizeof(int));
        s.global = malloc(s.n_total * sizeof(double));
        for (int p = 0, d = 0; p < s.size; p++) {
            s.counts[p] = base + (p < resto ? 1 : 0);
            s.displs[p] = d;
            d += s.counts[p];
        }
        printf("Processos: %d | Pontos: %d | Passos: %d | r = %.4f\n",
               s.size, s.n_total, s.n_passos, s.r);
    }
    MPI_Barrier(MPI_COMM_WORLD);

    double t1 = versao1(&s);
    double t2 = versao2(&s);
    double t3 = versao3(&s);

```

```
    if (s.rank == 0) {
        printf("V1 (Send/Recv)    : %.6f s\n", t1);
        printf("V2 (Isend/Irecv/Wait): %.6f s\n", t2);
        printf("V3 (sobreposicao) : %.6f s\n", t3);

        FILE *ft = fopen("dados/tempos.csv", "r");
        int novo = (ft == NULL);
        if (ft) fclose(ft);
        ft = fopen("dados/tempos.csv", "a");
        if (ft) {
            if (novo) fprintf(ft, "versao,processos,pontos,tempo_s\n");
            fprintf(ft, "V1,%d,%d,%.6f\n", s.size, s.n_total, t1);
            fprintf(ft, "V2,%d,%d,%.6f\n", s.size, s.n_total, t2);
            fprintf(ft, "V3,%d,%d,%.6f\n", s.size, s.n_total, t3);
            fclose(ft);
        }
        free(s.counts);
        free(s.displs);
        free(s.global);
    }

    MPI_Finalize();
    return 0;
}
```

Código 1: Simulação da barra 1D em anel com MPI (V1, V2 e V3).